

Sistema de Detecção de Semáforos Utilizando Processamento Digital de Imagens em Python.

PROPOSTA DE PROJETO — AVALIAÇÃO A3

Nome do(a) Aluno(a):	Anthony Felipe Fonseca de França
Matrícula:	12824130529 Turma: Noturno
Professor(a):	Karla Haryanna Santos Moura
Semestre:	2026.1 Data de Entrega: 11 / 06 / 2026

Nome do(a) Aluno(a):	Vitório Honorato De Freitas Pereira
Matrícula:	12824221916 Turma: Noturno
Professor(a):	Karla Haryanna Santos Moura
Semestre:	2026.1 Data de Entrega: 11 / 06 / 2026

1. Identificação do Projeto

Campo	Informação
Título do Projeto	Sistema de Detecção de Semáforos Utilizando Processamento Digital de Imagens em Python.
Disciplina	Computação Gráfica
Curso	Ciência da Computação
Tipo de Avaliação	A3 — Projeto Prático Individual
Semestre	2026.1
Linguagem	Python 3
Bibliotecas Principais	OpenCV, NumPy, Matplotlib
Plataforma	Google Colab

2. Introdução e Contextualização

A Computação Gráfica e o Processamento Digital de Imagens desempenham um papel fundamental no desenvolvimento de sistemas inteligentes capazes de interpretar informações visuais de forma automática. Entre suas diversas aplicações, destaca-se a visão computacional utilizada em veículos autônomos, sistemas de monitoramento e assistência à direção, permitindo que máquinas reconheçam objetos presentes no ambiente.

Nesse contexto, a identificação de semáforos é uma tarefa essencial para garantir a segurança e o correto funcionamento de sistemas de navegação autônoma. O reconhecimento preciso desses dispositivos permite que veículos interpretem sinais de trânsito e tomem decisões adequadas durante a condução.

Este projeto tem como objetivo desenvolver um sistema em Python capaz de detectar semáforos em imagens utilizando técnicas de processamento digital de imagens. Para isso, são empregadas operações de pré-processamento, detecção de bordas e identificação de formas geométricas, como linhas e círculos, permitindo localizar os elementos que compõem o semáforo.

Além de demonstrar conceitos estudados na disciplina de Computação Gráfica, o projeto evidencia como algoritmos de visão computacional podem contribuir para aplicações reais na área de mobilidade inteligente, servindo como base para futuras implementações em sistemas de veículos autônomos e assistência à direção.

3. Objetivos

3.1 Objetivo Geral

Desenvolver um sistema de processamento digital de imagens em Python capaz de identificar semáforos em imagens por meio de técnicas de visão computacional, utilizando operações de pré-processamento, detecção de bordas e reconhecimento de formas geométricas, com potencial aplicação em sistemas de veículos autônomos e assistência à condução.

3.2 Objetivos Específicos

- Capturar ou utilizar imagens contendo semáforos para realização dos testes do sistema.
- Implementar rotinas de pré-processamento das imagens, incluindo leitura, redimensionamento para 640×480, conversão para escala de cinza e normalização de pixels para o intervalo [0, 255].
- Realizar análise de histograma dos canais R, G e B para compreender a distribuição de cores da imagem.
- Aplicar técnicas de detecção de bordas para destacar os elementos presentes na imagem.
- Utilizar algoritmos de detecção de formas geométricas, como linhas e círculos, para localizar e identificar a estrutura do semáforo.
- Comparar a imagem original com a imagem processada, destacando os elementos detectados pelo algoritmo.
- Avaliar os resultados obtidos e discutir as possíveis aplicações do sistema em tecnologias de visão computacional para veículos autônomos.

4. Fundamentação Teórica

4.1 Imagem Digital

Uma imagem digital é uma representação de uma cena por meio de uma matriz de pixels, em que cada pixel armazena informações de intensidade luminosa ou de cor. Em imagens coloridas, normalmente é utilizado o modelo RGB (Red, Green e Blue), no qual a combinação dos três canais permite representar milhões de cores diferentes.

No contexto deste projeto, as imagens digitais representam cenas urbanas contendo semáforos, que serão analisadas por algoritmos de processamento de imagens. Cada pixel é utilizado como fonte de informação para que técnicas computacionais possam identificar características visuais, como bordas, formas e regiões de interesse.

4.2 Espaços de Cores

- Os espaços de cores representam diferentes formas de armazenar e interpretar as informações cromáticas de uma imagem digital. A escolha do espaço de cores adequado é fundamental para facilitar o processamento e a identificação de objetos por algoritmos de visão computacional.
- Neste projeto, a imagem original é representada no modelo BGR (padrão do OpenCV). Em seguida, é convertida para o espaço HSV (Hue, Saturation, Value), que separa a informação de cor da luminosidade, tornando a detecção de vermelho, amarelo e verde muito mais robusta a variações de iluminação.
- A escala de cinza também é utilizada para as etapas de detecção de bordas (Canny) e de linhas (Transformada de Hough), reduzindo a imagem a um único canal de intensidade luminosa.

4.3 Técnicas de Processamento de Imagens

As técnicas de processamento de imagens são utilizadas para extrair informações relevantes das imagens e facilitar a identificação de objetos. Neste projeto, foram empregadas operações de pré-processamento e algoritmos de visão computacional para detectar semáforos em imagens digitais.

Técnica	Descrição	Aplicação no Projeto
Redimensionamento	Padroniza a resolução da imagem para 640×480 usando cv2.INTER_AREA.	Normalização da entrada para processamento consistente.
Normalização	Ajusta os valores dos pixels para o intervalo [0, 255] via cv2.normalize().	Melhora o contraste e a robustez da detecção de cores.
Histograma de Cores	Plota a distribuição de intensidade dos canais B, G e R com cv2.calcHist().	Análise inicial da composição cromática da imagem.
Conversão HSV	Converte BGR para HSV separando cor (matiz) da luminosidade.	Base para a segmentação por cor do semáforo.
Máscaras de Cor	cv2.inRange() cria imagens binárias para vermelho, amarelo e verde.	Isolamento das luzes do semáforo.
Análise de Circularidade	Fórmula $4\pi \times \text{Área} / \text{Perímetro}^2$ avalia se os contornos são circulares.	Identificação das lâmpadas circulares do semáforo.
Detecção de Bordas (Canny)	Identifica mudanças bruscas de intensidade entre pixels.	Delimitação das estruturas presentes no semáforo.
Transformada de Hough para Linhas	Detecta segmentos de reta verticais na imagem.	Identificação do poste vertical do semáforo.

5. Metodologia

5.1 Tecnologia de Captura da Imagem

As imagens utilizadas neste projeto foram obtidas por meio de arquivos digitais contendo cenas urbanas com semáforos visíveis (arquivo: semaforo 2.jpeg). Essas imagens foram carregadas no ambiente Google Colab e processadas utilizando a biblioteca OpenCV, permitindo a aplicação das técnicas de visão computacional implementadas no sistema.

Após o carregamento, o algoritmo redimensiona a imagem para 640×480 pixels e aplica normalização, garantindo entradas padronizadas independente da câmera original. Em seguida, realiza a segmentação por cores HSV e a detecção de formas geométricas para identificar a estrutura do semáforo.

5.2 Ambiente de Desenvolvimento

Componente	Especificação
Linguagem	Python 3.10 ou superior
Ambiente de Execução	Google Colab / Jupyter Notebook
Biblioteca Principal	OpenCV-Python (cv2) versão 4.x
Visualização	Matplotlib 3.x
Manipulação Numérica	NumPy 1.x

5.3 Pipeline de Processamento

- Aquisição: leitura da imagem com cv2.imread() — arquivo semaforo 2.jpeg
- Pré-processamento: redimensionamento para 640×480 (cv2.INTER_AREA) e normalização para [0, 255] (cv2.NORM_MINMAX)

- Análise de Histograma: cálculo e exibição da distribuição de intensidades dos canais B, G e R com `cv2.calcHist()`
- Conversão de Espaço de Cor: BGR → HSV para segmentação robusta por cor
- Segmentação por Cor: máscaras binárias para vermelho (duas faixas HSV), amarelo e verde via `cv2.inRange()`
- Detecção de Contornos e Circularidade: `cv2.findContours()` + análise de circularidade ($4\pi \times A/P^2$) para identificar lâmpadas circulares
- Cálculo de Centros: identificação dos centros geométricos dos círculos detectados e verificação de alinhamento vertical
- Detecção de Bordas: conversão para cinza, `GaussianBlur` e operador Canny (limiares 50/150) com dilatação do resultado
- Transformada de Hough: `HoughLinesP` para detectar linhas verticais (estrutura do poste)
- Caixa Delimitadora e Resultado Final: retângulo sobre as linhas verticais detectadas + contagem de círculos dentro da caixa

6. Estrutura de Implementação

6.1 Instalação das Dependências

```
pip install opencv-python numpy matplotlib
```

6.2 Funções Principais

Função	Finalidade	Biblioteca
carregar_imagem(path)	Lê, redimensiona (640×480) e normaliza a imagem	OpenCV + NumPy
analisar_histograma(img)	Plota distribuição de intensidades por canal B, G, R	Matplotlib + OpenCV
criar_mascaras_hsv(img)	Gera máscaras binárias para as 3 cores do semáforo	OpenCV
verificar_circulo(cnt, img)	Calcula circularidade e desenha contornos circulares	OpenCV + NumPy
extrair_centros(contornos)	Retorna centros dos círculos válidos para análise de alinhamento	OpenCV
detectar_linhas_verticais(img)	Canny + dilatação + <code>HoughLinesP</code> filtrando linhas verticais	OpenCV
detectar_semaforo()	Integra resultados: caixa delimitadora + contagem de círculos	OpenCV

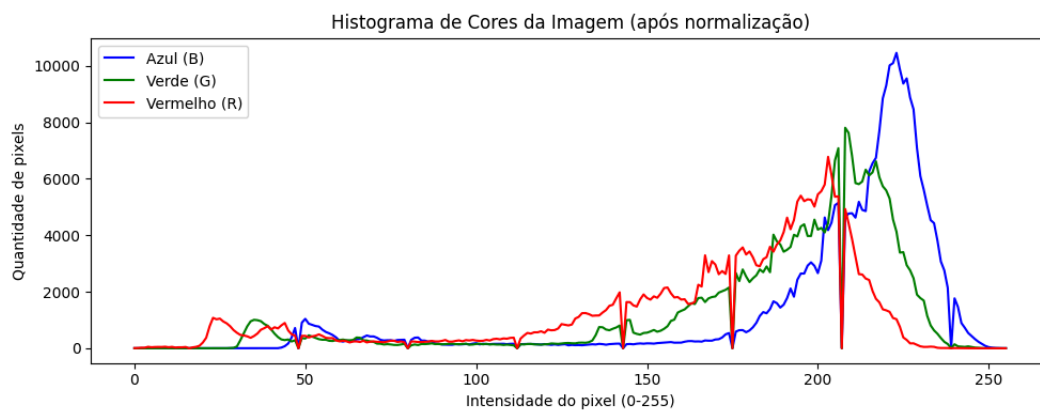
7. Resultados Esperados

Ao final da execução, o sistema deverá apresentar os seguintes entregáveis visuais e analíticos:

- Imagem original exibida após redimensionamento para 640×480 e normalização, com confirmação no console

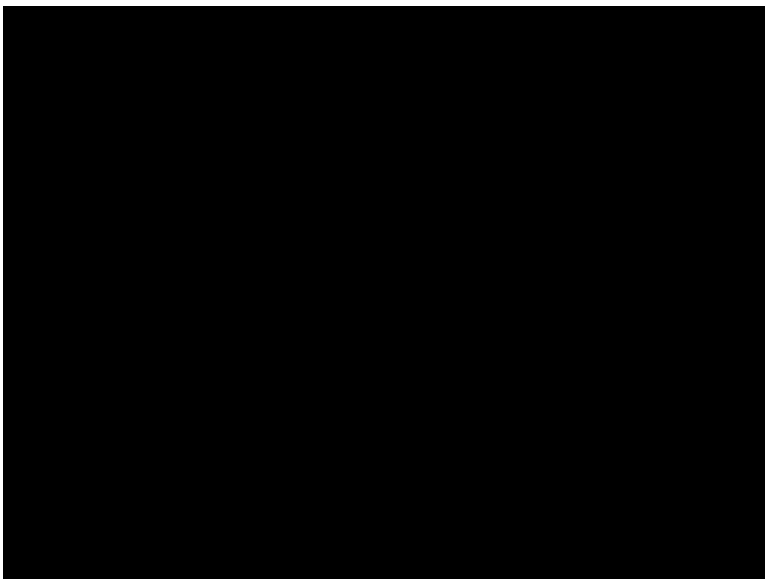


- Histograma dos canais B, G e R da imagem normalizada com análise visual da distribuição de cores

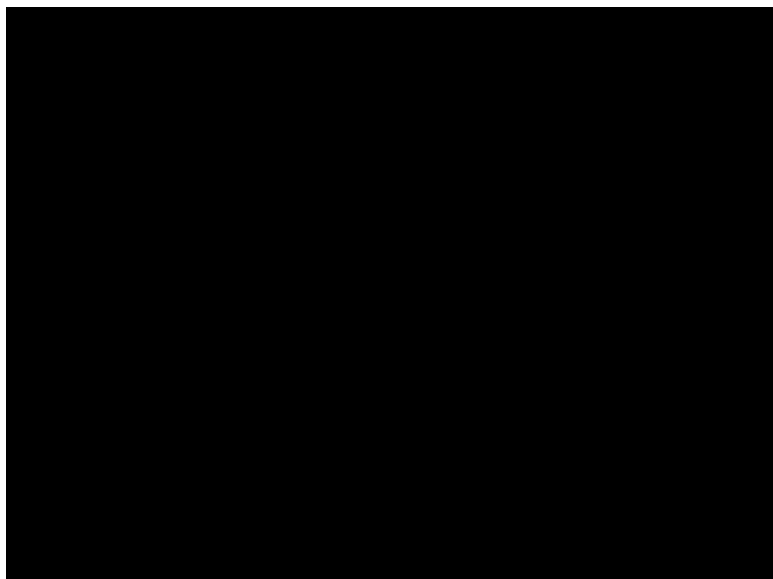


- Máscaras binárias das cores vermelho, amarelo e verde segmentadas no espaço HSV

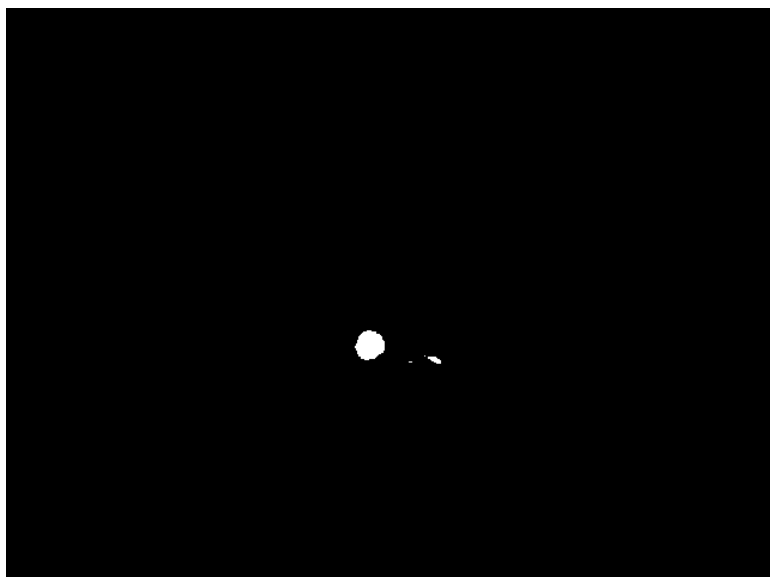
Máscara Vermelha:



Máscara Amarela:



Máscara Verde:



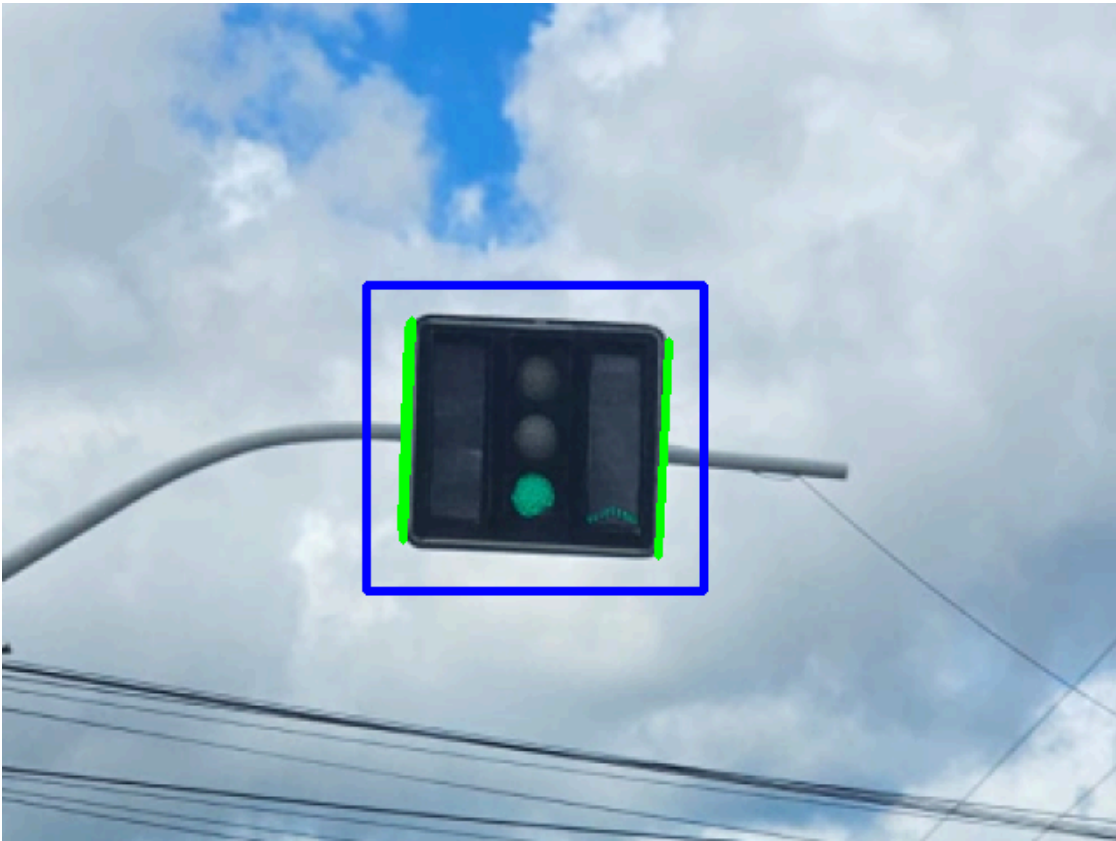
- Imagem com contornos circulares detectados desenhados sobre as regiões de cor



- Imagem com linhas verticais detectadas pela Transformada de Hough destacadas em verde



- Caixa delimitadora azul ao redor da região do possível semáforo com número de círculos detectados



- Aviso na imagem comunicando se tem um círculo dentro do retângulo azul



- Saída textual indicando  SEMÁFORO DETECTADO ou  Nenhum semáforo detectado



8. Cronograma de Desenvolvimento

Fase / Atividade	Prazo Sugerido	Responsável
Definição do tema e escolha da imagem	Semana 1	Aluno
Origem documentação fotográfica	Semana 1–2	Aluno
Pesquisa bibliográfica e setup do ambiente	Semana 2	Aluno
Implementação do pré-processamento (resize + normalização + histograma)	Semana 3	Aluno
Segmentação HSV, circularidade e centros	Semana 3–4	Aluno
Deteção de bordas, Hough e resultado final	Semana 4	Aluno
Escrita do relatório técnico	Semana 5	Aluno
Revisão, entrega e apresentação oral	Semana 6	Aluno

9. Critérios de Avaliação

Critério de Avaliação	Peso (%)	Observações
Documentação da origem da imagem	15%	Deve evidenciar participação ativa do aluno
Qualidade e variedade das técnicas	30%	Mínimo de 4 técnicas distintas implementadas
Correção e funcionamento do código	5%	Código deve executar sem erros
Análise crítica dos resultados	10%	Discussão técnica dos efeitos aplicados
Relatório técnico escrito	10%	Formatação, clareza e referências bibliográficas
Apresentação oral / defesa	30%	Domínio do conteúdo e clareza expositiva

10. Entregáveis

O aluno deverá submeter, em pasta compactada (.zip) ou repositório Git (GitHub/GitLab), os seguintes artefatos:

- projeto_a3_<nome>.ipynb — código Python completo e comentado (Google Colab)
- semaforo 2.jpeg — imagem utilizada no projeto
- resultados/ — pasta contendo as imagens geradas pelo processamento
- Artigo_a3.pdf — este documento preenchido com os resultados reais obtidos
- README.md — instruções de execução (dependências, versões Python, como rodar)
- [Opcional] video_demo.mp4 — demonstração em vídeo da execução (máx. 5 minutos)

11. Referências Bibliográficas

- [1] GONZALEZ, R. C.; WOODS, R. E. Processamento Digital de Imagens. 3. ed. São Paulo: Pearson, 2010.
- [2] BRADSKI, G.; KAEHLER, A. Learning OpenCV 4: Computer Vision with Python 3. O'Reilly Media, 2019.
- [3] SZELISKI, R. Computer Vision: Algorithms and Applications. 2. ed. Springer, 2022.
- [4] OpenCV Documentation. Disponível em: <https://docs.opencv.org/> (acesso em 2026).
- [5] NumPy Documentation. Disponível em: <https://numpy.org/doc/> (acesso em 2026).
- [6] Matplotlib Documentation. Disponível em: <https://matplotlib.org/stable/contents.html> (acesso em 2026).

12. Assinaturas e Validação

Assinatura do(a) Aluno(a)

Nome: Anthony Felipe Fonseca de França

Data: 11 / 06 / 2026

Assinatura do(a) Professor(a) Orientador(a)

Nome: Karla Haryanna Santos Moura

Nota: _____

Assinatura do(a) Aluno(a)

Nome: Vitório Honorato De Freitas Pereira

Data: 11 / 06 / 2026

Assinatura do(a) Professor(a) Orientador(a)

Nome: Karla Haryanna Santos Moura

Nota: _____